Obstacle-Avoidable Robotic Motion Planning Framework Based on Deep Reinforcement Learning

Huashan Liu, *Senior Member, IEEE*, Fengkang Ying, Rongxin Jiang, Yinghao Shan, *Member, IEEE*, and Bo Shen, *Senior Member, IEEE*

***Abstract*—Although robotic trajectory generation has been extensively studied, the motion planning in environments with obstacles still faces some open issues and is yet to be explored. In this article, a universal motion planning framework based on deep reinforcement learning (DRL) is proposed to achieve autonomous obstacle avoidance for robotic tasks. First, a prophet-guided actor–critic structure based on the expert strategy is designed, which can realize prompt replanning when the task scenario changes. Second, an expansive dual-memory sampling mechanism is proposed to efficiently augment expert data from only a few demonstrations. It also improves the training efficiency of DRL algorithms through an increasingly unbiased sampling rule. Third, a composite obstacle-avoidable reward system is designed to achieve collision-free motion for both a robot's end effector and its body/link. It can build a dense reward map, and strike a balance between obstacle avoidance and action exploration. Finally, experimental results have validated the performance of the proposed work in three different scenes.**

***Index Terms*—Composite obstacle-avoidable reward (COR), deep reinforcement learning (DRL), expansive dual-memory sampling (EDS), prophet-guided actor–critic (PAC), robotic motion planning.**

Manuscript received 11 April 2023; revised 17 June 2023, 13 September 2023, and 8 November 2023; accepted 5 March 2024. Date of publication 28 March 2024; date of current version 17 December 2024. Recommended by Technical Editor D. Dong and Senior Editor N. G Tsagarakis. This work was supported in part by the National Natural Science Foundation of China under Grant 62373095, Grant 62303107, and Grant 62273088; in part by the Natural Science Foundation of Shanghai under Grant 21ZR1401100; and in part by the Discipline Innovation Cultivation Program under Grant XKCX202304. *(Huashan Liu and Fengkang Ying contributed equally to this work.) (Corresponding author: Yinghao Shan.)*

Huashan Liu, Rongxin Jiang, Yinghao Shan, and Bo Shen are with the College of Information Science and Technology, Donghua University, Shanghai 201620, China, and also with the Engineering Research Center of Digitized Textile & Apparel Technology, Ministry of Education, Donghua University, Shanghai 201620, China (e-mail: hsliu@dhu.edu.cn; rxjiang@mail.dhu.edu.cn; shanyh@dhu.edu.cn; bo.shen@dhu.edu.cn).

Fengkang Ying is with the Integrative Sciences and Engineering Programme, NUS Graduate School, National University of Singapore, Singapore 119077, and also with the Advanced Robotics Centre, National University of Singapore, Singapore 117608 (e-mail: fengkang@u.nus.edu).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/TMECH.2024.3377002.

Digital Object Identifier 10.1109/TMECH.2024.3377002

I. Introduction

A. Motivations and Related Works

WITH the vigorous development of industry, robots have been increasingly applied to various situations. In practice, robots are usually required to have the ability for obstacle avoidance, so as to autonomously and safely work in and adapt to complicated or unstructured environments [1]. To enable robots to complete the predetermined task under the premise of obstacle avoidance, various methods have been investigated to solve the problem of obstacle avoidance.

However, traditional robotic motion planning methods with obstacle avoidance still have some problems. First, although the traditional path planning methods, such as rapidly exploring random tree (RRT), probabilistic roadmap (PRM), are probabilistically complete, they converge to nonoptimal values [2], [3]. Even though asymptotically optimal methods such as RRT* and PRM* have been proposed to solve the optimization problem by constantly increasing the number of nodes and iteration, their optimization ability is not strong enough due to inadequate exploitation of previous iteration experience [4]. Second, conventional inverse kinematics (IK) mapping from Cartesian space to joint space is complicated and only valid for a certain robot structure, especially for manipulators with redundant degrees of freedom (DoFs) [5]. In addition, traditional motion planning methods are usually highly customized, and replanning is needed even when there is only a small change in the task [6].

Recently, the intersection of deep reinforcement learning (DRL) and robotics has yielded fresh insights into robotics-related problem, which enables robots to execute versatile tasks that are arduous to be realized by conventional methods. In [7], DRL has been used to determine the manipulation sequence for a pick-and-place task. In [8], an IK-free trajectory generation scheme for multiprocess robotic assignments has been designed based on deep deterministic policy gradient (DDPG). DRL has also been adopted in [4] to enhance the ability for trajectory optimization of traditional path planners such as RRT*, which was verified by an obstacle avoidance task on a 7-DoF manipulator. Moreover, although it is formidable to control soft robots by conventional methods, two advanced DRL methods: twin delayed DDPG (TD3) and soft actor–critic (SAC), have been validated as effective in controlling a soft robot after ten million

1083-4435 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

of training iterations [9]. In addition, DRL has been shown beneficial for the compliant control of robots. For example, DDPG has been utilized in [10] to realize a peg-in-hole assembly according to the observed contact force, based on which, a hybrid model that combines DRL with imitation learning has been proposed in [11] to achieve a similar task with a higher learning efficiency. DRL has even been leveraged to handle deformable manipulation that is extremely challenging, such as the cloth unfolding task [12].

Despite DRL's strong ability to help solve robotics-related problems, there are still some issues awaiting to be addressed.

First, it is a widely acknowledged deficiency that the training environment and the testing environment should be exactly the same for DRL methods, and retraining is always required if the task scene is altered. However, whether DRL methods based on deterministic policy gradient such as DDPG [13] and TD3 [14], or those based on stochastic policy such as SAC [15], poor policy decision in early training is a common problem that leads to slow policy optimization speed [16]. The aforementioned drawbacks seriously influence the retraining efficiency of DRL in response to a new task scene. For the robotic obstacle avoidance tasks, the layout, number, and shape of obstacles vary in different task scenes. Although a DRL-based method has been proposed in [17] for path planning with a dynamic obstacle, the task scene is quite basic with only one obstacle moving in a restricted area. There is no guarantee that the well-trained model can be directly applied to other task scenes with multiple obstacles or different working areas, and retraining is still required. But even for such a basic scene, it took more than 100 000 episodes to train the DRL. Thus, great time cost can be foreseen in response to the change of task scene without a prompt retraining approach for DRL.

Second, unlike RRT and PRM, DRL algorithms have an experience replay mechanism that improves data utilization, and methods such as prioritized experience replay [11], [17] and hindsight experience replay [18], [19] have been used to improve training efficiency. However, these sampling methods are also of limited use if the high-quality data are difficult to be obtained. Meanwhile, the large number of low-quality data stored in the memory when the robot touches obstacles in the early training period also lead to inefficient learning. To solve this problem, expert teaching is often used to obtain high-quality expert experience data [20]. But it is of high cost for robots to obtain sufficient data from human experts. In addition, policy learning based on only a small number of expert data may be restricted [21], making it still difficult for DRL to find the solutions of motion planning.

Third, the design of reward function will directly affect the action exploration ability of DRL and the direction of policy optimization [22]. Particularly, for robotic obstacle avoidance tasks, it is important to achieve a balance between obstacle avoidance and action exploration. The obstacle-avoidable reward has been designed on the basis of a penalty mechanism, which is inversely proportional to the distance between the end effector and obstacles. However, this may create a virtual penalty barrier that wraps around the obstacle, hindering the exploration, especially when there are concave polyhedron obstacles between the robot and target. In some other work, sparse reward is given when the end effector approaches the target point, or collides with obstacles [7], [23], [24]. Nevertheless, it may cause a lack of positive rewards, making the agent difficult to optimize its policy and trap into a local optimum. In addition, many of the existing trajectory generation methods, such as RRT, RRT*, PRM, PRM*, and DRL-based ones in [4] and [25], only focus on the obstacle avoidance of the tool center point (TCP). However, the trajectory in the workspace should then be mapped to the joint space by IK to facilitate the motion control of manipulators. Hence, even though the TCP moves in a collision-free trajectory, the body/link of the manipulator may still collide with obstacles.

B. Contributions and Organization of This Article

In order to respond to the aforementioned discussions, we aim to provide a general framework to handle the trajectory generation problem to achieve autonomous and sufficient obstacle-avoidable motion planning for robotic tasks based on DRL. The main contributions can be summarized as follows.

1) A prophet-guided actor–critic (PAC) structure is designed for efficient policy learning in a new scene, by integrating the policies trained in other similar scenes as prophets. It can strongly support the prompt retraining of DRL, making it adapt to the changes in the task scene quickly. It is a general framework compatible with all the DRL algorithms based on the actor–critic structure, and can achieve efficient policy transfer among different but similar scenarios.
2) An expansive dual-memory sampling (EDS) mechanism is proposed, which includes a novel method to expand expert memory by DRL-integrated generative adversarial imitation learning (xGAIL) and an increasingly unbiased dual-memory sampling strategy. Massive high-quality expert data can be augmented based on a very small number of expert demonstrations. By hybrid sampling between the two memories, the agent can learn more efficiently.
3) A composite obstacle-avoidable reward (COR) is proposed, which focuses on collision-free movement of the end effector and the body of a robot simultaneously to ensure safety. The method comprehensively considers robot's pose error and the concept of the artificial potential field (APF), which includes an attraction reward and a repulsion reward to make the robot learn to reach the target while avoiding obstacles. It can help the agent to generate an optimal trajectory quickly, and achieve a good balance between obstacle avoidance and action exploration in a complex environment.

The rest of this article is organized as follows. Section II describes the structure of the proposed PAC. Section III presents the details of the proposed EDS. The design of the COR is described in Section IV. Section V introduces the overall implementation of the obstacle-avoidable motion planning method. Experimental comparisons are given in Section VI. Finally, Section VII concludes this article.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

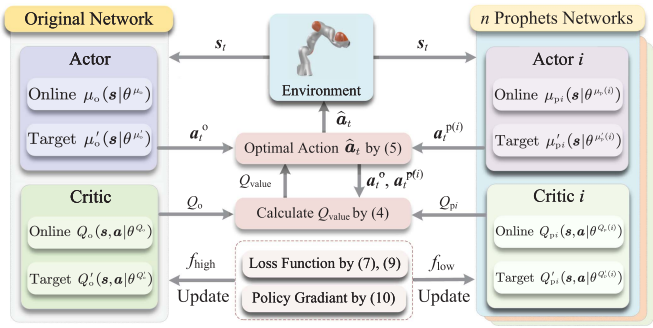


Fig. 1. Framework of the PAC.

II. Fundamentals of Algorithm

In this section, the problem of the obstacle-avoidable motion planning for robotic tasks based on DRL is formulated, and the structure of the proposed prophet-guided actor–critic (PAC) algorithm is illustrated.

A. Problem Formulation

The obstacle-avoidable trajectory generation problem can be described as a Markov decision process and be solved by DRL-based methods. At each time step t, the action a_t is generated by the agent according to the current state s_t. Then, the state is transferred to s_{t+1}, and a reward r_t is received as feedback of the state–action pair.

Given that the output of the exploration driven by DRL is a continuous action of the robot, each pose of which corresponds to complex model-based IK solving in existing work [26], in contrast, we are committed to adopting an IK-free method to make the proposed approach irrelevant to the IK model. That is, let DRL control the joint angles directly, and thus, the action $\boldsymbol{a}_t$ is defined as

$$\boldsymbol{a}_t = [\Delta\theta_1, \Delta\theta_2, \ldots, \Delta\theta_i] \tag{1}$$

where $\Delta\theta_i$ is the angular increment of the ith joint.

The state $\boldsymbol{s}_t$ is defined as

$$\boldsymbol{s}_t = [\boldsymbol{A}_t, \boldsymbol{p}_0, \boldsymbol{p}_1, \boldsymbol{E}, D_s, d_t] \tag{2}$$

where $\boldsymbol{A}_t$ is the current joint angles of the manipulator. $\boldsymbol{p}_0 = (x_0, y_0, z_0)$ and $\boldsymbol{p}_1 = (x_1, y_1, z_1)$ are the current position of the target and the TCP relative to the world coordinate system, respectively. $\boldsymbol{E} = [\Delta x, \Delta y, \Delta z, \Delta\alpha, \Delta\beta, \Delta\gamma]$ represents the position and orientation deviations between the TCP and the target. D_s is the shortest distance between the obstacle and the robot. d_t is a Boolean value used to determine whether the TCP reaches the target area within the allowable errors.

B. Prophet-Guided Actor–Critic (PAC)

PAC is proposed as a general framework that can be used for DRL algorithms based on the actor–critic structure, such as DDPG, TD3, SAC, etc. In this subsection, the implementation of the proposed PAC policy learning is described.

As shown in Fig. 1, the proposed PAC consists of an original network and multiple prophet networks, and they can have different network structures. For a complex obstacle-avoidable trajectory generation task, partial or the whole obstacles can be first removed to simplify the task and form several similar scenes, and the prophet networks are used to train these simplified scenes, where successful experiences can be easily gained. Then, the trained prophets can be used to guide the training of the original complex task by the original network.

For convenience, DDPG is used as an example to describe the proposed general framework PAC in detail. Note that PAC does not change the basic setups of DRL algorithms, thus it can be applied to TD3 and SAC in similar ways to DDPG.

Define i $(i = 1, 2, \ldots, n)$ as the number of prophets incorporated, then at every time step t, for state $\boldsymbol{s}_t$, the original online actor network (denoted as $\mu_o(\boldsymbol{s}_t|\theta^{\mu_o})$ with parameter θ^{μ_o}) and the prophet online actor networks (denoted as $\mu_{pi}(\boldsymbol{s}_t|\theta^{\mu_p(i)})$ with parameter $\theta^{\mu_p(i)}$) generate actions $\boldsymbol{a}_t^o$ and $\boldsymbol{a}_t^{p(i)}$, respectively,

$$\begin{cases} \boldsymbol{a}_t^o = \mu_o(\boldsymbol{s}_t|\theta^{\mu_o}) + \mathcal{N}_n \\ \boldsymbol{a}_t^{p(i)} = \mu_{pi}(\boldsymbol{s}_t|\theta^{\mu_p(i)}) + \mathcal{N}_n \end{cases} \tag{3}$$

where $\mathcal{N}_n$ is the Ornstein–Uhlenbeck noise [13].

Original online critic network (denoted as $Q_o(\boldsymbol{s}_t, \boldsymbol{a}_t|\theta^{Q_o})$ with parameter θ^{Q_o}) and prophet online critic networks (denoted as $Q_{pi}(\boldsymbol{s}_t, \boldsymbol{a}_t|\theta^{Q_p(i)})$ with parameter $\theta^{Q_p(i)}$) evaluate the action by the comprehensive evaluation Q_{value}

$$Q_{value}(\boldsymbol{s}_t, \boldsymbol{a}_t) = (1-\eta)Q_o + \sum \eta Q_{pi} \tag{4}$$

where $\eta = 0.5e^{-t/T}$, t is the current accumulated training steps, and T is a constant. In the early training, the original network has random parameters. Thus, the Q values given by prophets play a major role in this period.

Then, the optimal action $\hat{\boldsymbol{a}}_t$ [16] is defined as

$$\hat{\boldsymbol{a}}_t = \underset{\boldsymbol{a}_t \in \{\boldsymbol{a}_t^o, \boldsymbol{a}_t^{p(i)}\}.}{\arg\max} Q_{value}(\boldsymbol{s}_t, \boldsymbol{a}_t) \tag{5}$$

In DRL, the Bellman equation is used to calculate the value expectation Q_{target}. According to the Bellman equation, Q_{target} is defined as

$$Q_{target} = r_t + \gamma Q'(\boldsymbol{s}_{t+1}, \mu'(\boldsymbol{s}_{t+1}|\theta^{\mu'})\theta^{Q'}) \tag{6}$$

where γ is the discount factor, μ' is the target actor network with parameter $\theta^{\mu'}$, and Q' is the target critic network with parameter $\theta^{Q'}$.

For the online critic network, the temporal difference error L_1 is defined as

$$L_1 = \frac{1}{N}\sum (Q_{target} - Q(\boldsymbol{s}_t, \boldsymbol{a}_t|\theta^{Q_j}))^2 \tag{7}$$

where N is the batch size, and $\theta^{Q_j} \in \{\theta^{Q_o}, \theta^{Q_p(i)}\}$. Note that TD3 and SAC have a dual-Q structure, and the minimum $Q(\boldsymbol{s}_t, \boldsymbol{a}_t)$ is chosen here. The prophet auxiliary loss function L_2 is defined as

$$L_2 = \frac{1}{N}\sum ||Q_o - Q_p||^2 \tag{8}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

where $||\cdot||$ denotes the Euclidean norm, and Q_{p} is the average of the $Q_{\mathrm{p}i}$ output by all the prophets incorporated. L_2 is designed to enhance the training stability in a multiactor–multicritic structure according to [27]. Meanwhile, this term enables the original critic network to imitate the behaviors of the prophets.

Thus, the original online critic network is optimized by minimizing the composite loss function L as

$$L = \lambda_1 L_1 + \lambda_2 L_2 \tag{9}$$

where λ_1 and λ_2 are the weight coefficients. Meanwhile, the prophet online critic networks are updated merely by L_1 at a low frequency, considering L_2 may make them imitate the original critic network.

In addition, the actor networks are updated by the gradient

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{\boldsymbol{s}_t \sim \rho^\theta} \left[\left(\nabla_{\boldsymbol{a}} Q\left(\boldsymbol{s}_t, \boldsymbol{a}_t; \theta^Q\right) \nabla_{\theta^\mu} \mu\left(\boldsymbol{s}_t; \theta^\mu\right) \right) \right] \tag{10}$$

where ρ^θ represents the distribution of $\boldsymbol{s}_t$, and J [13] is the mathematical expectation for discounted future cumulative reward R_t, where $R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{n-t} r_n$.

Besides, the parameters of original target networks and prophet target networks are optimized by soft update [16].

Remark 1: Overall, in order to realize prompt replanning in a more complex scenario, PAC shows unique features and advantages compared with sequential transfer learning (STL) [28]. STL uses a pretrained network in a similar scene and retrains it with the new target task. However, the pretrained network parameters disappear quickly in the early retraining phase. In some complex tasks, this would cause divergence of the pretrained policies and result in extremely low rewards, which may impact the training efficiency if it lasts for a long time. In comparison, the proposed PAC adopts a delayed update mechanism. As shown in Fig. 1, for each time step t, the update frequency of the original network is defined as f_{high}, while that of the prophet networks is defined as $f_{\mathrm{low}} = f_{\mathrm{high}}/n_f$, where $n_f \geq 2$ is a positive integer. Moreover, initially, the original network can copy the prophet's parameters. Such a mechanism with a lower update frequency is beneficial to the retention of expert strategy in the early training period.

Remark 2: Second, STL can only refer to one pretrained model for expert strategy. However, the single model chosen does not always provide a proper output during the whole training process with another scene. Moreover, it is sometimes difficult to manually select and determine the best pretrained model. In comparison, the proposed PAC forms a unique multiactor–multicritic structure that can refer to and incorporate multiple pretrained models in various similar tasks to cosupervise the original network's training. Multiple actions can be output each time to enhance the action exploration, and the best action is then chosen according to the fairer evaluation made by (4). That is to say, the prophet networks act as experts to guide the original network to acquire high-reward actions in the early training, and then, evolve as its colleagues to enhance the overall performance of DRL. Such a colleague relationship has been proven powerful for DRL [16], and the proposed transition mechanism from expert to colleague relationship allows policies to generalize efficiently among similar scenarios.

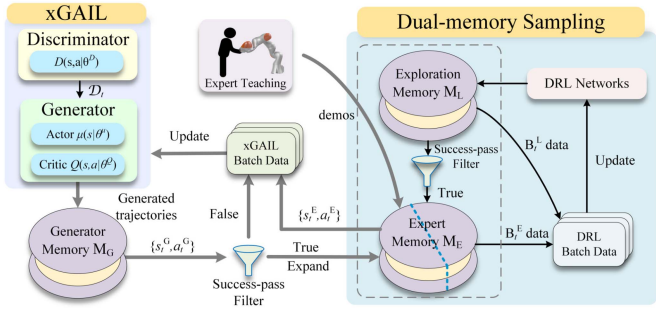


Fig. 2. Framework of the EDS. Expert memory expansion by xGAIL and increasingly unbiased dual-memory sampling in EDS are two sequential processes. The former expands the demonstration data to initially populate the expert memory $\mathrm{M_E}$, and the latter updates it by incorporating the filtered data from exploration.

Remark 3: Expert strategies chosen by humans are frequently defaulted to be the optimal strategy for imitation learning, which may produce suboptimal results. However, the quality of the expert strategy selected by humans cannot be guaranteed and is always not optimal, which directly influences the learning result. In this article, the prophet networks act as general self-evolving experts with the ability to slowly and continuously optimize the strategy, so as to be satisfying colleagues of the original network as training proceeds. They are highly applicable for diverse scenes and complex tasks, where human demonstrations are difficult to be obtained.

III. Expansive Dual-Memory Sampling (EDS)

Expert-guided policy learning has been proven to be effective to expedite the training especially in the early stages, where the expert demonstrations can be generated in various ways, such as teaching and playback, traditional IK-based trajectory planning, etc. [16]. However, such manually generated expert demonstrations cannot guarantee to be optimal. To solve such problems, in this section, the expansion of expert memory based on generative adversarial imitation learning (GAIL) is described, and a dual-memory sampling rule for the expert-guided policy learning strategy is also designed.

A. Expert Memory Expansion Based on DRL-Integrated Generative Adversarial Imitation Learning (xGAIL)

In this article, a general structure that can integrate arbitrary DRL algorithm with GAIL (denoted as xGAIL) is proposed to expand expert memory with self-optimized data, as expressed by Algorithm 1.

As shown in Fig. 2, the generator in xGAIL is a DRL algorithm with an actor–critic structure, which generates action $\boldsymbol{a}_t$ based on the current state $\boldsymbol{s}_t$. The discriminator, formulated as $D(\boldsymbol{s}_t, \boldsymbol{a}_t|\theta^D)$ with parameter θ^D, is mainly used to discriminate the similarity between the current action by the agent and the one by the expert. $D(\boldsymbol{s}_t, \boldsymbol{a}_t|\theta^D)$ has the output $\mathcal{D}_t$, which takes the value [0, 1] at time step t. The closer $\mathcal{D}_t$ to 1, the higher

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

the similarity between the agent action and the expert action is. Hence, using $\mathcal{D}_t$ as a reward function enables the agent to learn in the direction of the expert policy.

The expert policy is defined as π_{E}, and the trajectory sample set T_{E} obtained by π_{E} is defined as

$$T_{\mathrm{E}} = \{\tau_1, \tau_2, \ldots, \tau_i | \pi_{\mathrm{E}}\} \tag{11}$$

where τ_i represents the ith trajectory sample. Each trajectory τ_i can be further split into a sequential finite set of state–action pairs $(\boldsymbol{s}_t^{\mathrm{E}}, \boldsymbol{a}_t^{\mathrm{E}})$.

In order to clearly distinguish the expert policy from the original policy, the objective of the discriminator $D(\boldsymbol{s}_t, \boldsymbol{a}_t|\theta^D)$ is to minimize

$$\begin{aligned}\nabla_\theta^D D|_{\boldsymbol{s}_i,\boldsymbol{a}_i} = &\hat{E}_{T_{\mathrm{G}}}[\nabla_\theta^D \log(D(\boldsymbol{s}_i, \boldsymbol{a}_i))] \\ &+ \hat{E}_{T_{\mathrm{E}}}[\nabla_\theta^D log(1 - D(\boldsymbol{s}_i, \boldsymbol{a}_i))]\end{aligned} \tag{12}$$

where T_{G} is the sample set obtained from the generator.

The optimization objective of the generator is almost the same as the integrated DRL algorithm, that is, optimizing the actor and critic networks to produce actions that are as close as possible to the expert policy, except the only difference of using r_t^{G} to replace r_t in (6)

$$r_t^{\mathrm{G}} = \mathcal{D}_t. \tag{13}$$

By the proposed xGAIL, only a small number of expert demonstrations are needed in the initial stage, since abundant expert data can be automatically produced as described by Algorithm 1. However, the expert demonstrations may still fail the given task of avoiding obstacles. To maximize the effect of xGAIL, it is more preferable to ensure the success of the given demonstrations, although the failed ones can also help to improve the learning efficiency to some extent. Note that, to reach the Nash equilibrium, the generator and the discriminator in xGAIL should be trained for a certain number of times, and when training one of them, the parameters of the other must keep unchanged. In addition, a trained generator can also be reused as one of the prophets in the proposed PAC. For each interaction, the proposed PAC will autonomously decide which prophet provides the optimal action by (5).

Algorithm 1: xGAIL.

```
Initialize generator (actor network μ_G(s_t|θ^{μ_G}), critic
  network Q_G(s_t, a_t|θ^{Q_G})) and discriminator
  D(s_t, a_t|θ^D), respectively;
Initialize θ^{Q_i}, θ^μ and N_n for xGAIL;
Initialize target networks: θ^{Q'_G} ← θ^{Q_G}, θ^{μ'_G} ← θ^{μ_G};
Store expert teaching demonstrations to expert
  memory M_E;
for each episode do
    Randomly reset the environment.
    for each step do
        Select action by a_t = μ_G(s_t; θ^{μ_G}) + N_n;
        Execute a_t and observe s_{t+1}, r_t^G, d_t;
        Store (s_t, a_t) to generator memory M_G;
    Sample N data from M_G and M_E by (14);
    Update the discriminator network;
    Update the generator network;
    Optimize the target networks μ'_G(s_{t+1}|θ^{μ'_G}) and
      Q'_G(s_{t+1}, a_{t+1}|θ^{Q'_G}) by soft update.
```

B. Increasingly Unbiased Dual-Memory Sampling

In the early training period, the data stored in the memory have low quality, leading to inefficient learning. It is even more difficult for robots to obtain memories with high rewards when there are obstacles in the environment. To improve this situation, a dual-memory structure is constructed. As shown in Fig. 2, an exploration memory $\mathrm{M_L}$ is used to store the memory data generated by DRL networks, while an expert memory $\mathrm{M_E}$ is used to store expert data. The latter is expanded by a success-pass filter to winnow the successful samples from $\mathrm{M_L}$ into $\mathrm{M_E}$.

In order to improve the learning efficiency of DRL algorithms, an increasingly unbiased dual-memory sampling rule is designed to guide the agent's policy learning. At time step t, B_t^{L} and B_t^{E} data are randomly sampled from exploration memory $\mathrm{M_L}$ and expert memory $\mathrm{M_E}$, respectively,

$$\begin{cases} B_t^{\mathrm{L}} = \lfloor \frac{N}{2} + \frac{t}{\mathrm{T_s}} \rfloor, B_t^{\mathrm{E}} = N - B_t^{\mathrm{L}}, & 0 \le t < \frac{N\mathrm{T_s}}{2} \\ B_t^{\mathrm{L}} = N, B_t^{\mathrm{E}} = 0, & t \ge \frac{N\mathrm{T_s}}{2} \end{cases} \tag{14}$$

where N is the batch size sampled from the memory for each time step t, and $\mathrm{T_s}$ denotes the attenuation step. Both B_t^{L} and B_t^{E} are nonnegative integers.

To make the sampling increasingly unbiased, B_t^{E} decreases by every $\mathrm{T_s}$ steps, which gradually reduces the sampling weight of the data from the expert memory $\mathrm{M_E}$ until DRL samples all the transitions from the exploration memory $\mathrm{M_L}$.

IV. Composite Obstacle-Avoidable Reward (COR)

In this section, a composite reward system composed of reward terms with respect to the pose of the TCP and the APF-like concept is proposed, as shown in Fig. 3.

In order to reduce the sparsity of the reward, the pose reward function r_{pose}, which can provide a reward value for the pose error of the TCP at each time step t, is defined as

$$\begin{aligned} r_{\mathrm{pose}} = &-||\boldsymbol{p}_1 - \boldsymbol{p}_0|| \\ &- \varphi(|\alpha_1 - \alpha_0| + |\beta_1 - \beta_0| + |\gamma_1 - \gamma_0|) + \delta \end{aligned} \tag{15}$$

where $\boldsymbol{p}_0 = (x_0, y_0, z_0)$ and $\boldsymbol{p}_1 = (x_1, y_1, z_1)$ are the current position of the target and the TCP relative to the world coordinate system. $(\alpha_0, \beta_0, \gamma_0)$ and $(\alpha_1, \beta_1, \gamma_1)$ are the Tait-Bryan angles of the target and the TCP, and φ is a weight factor used to adjust the importance of position and orientation accuracy to the current task. Also, additional coefficients can be assigned to each component of the Tait-Bryan angles, as probably the control of only one or two of them is needed in some other tasks. δ equals 1 if the TCP reaches the target point with a proper orientation, and otherwise 0.

Generally, for the robotic task of obstacle-avoidable trajectory generation, obstacle avoidance and action exploration usually

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

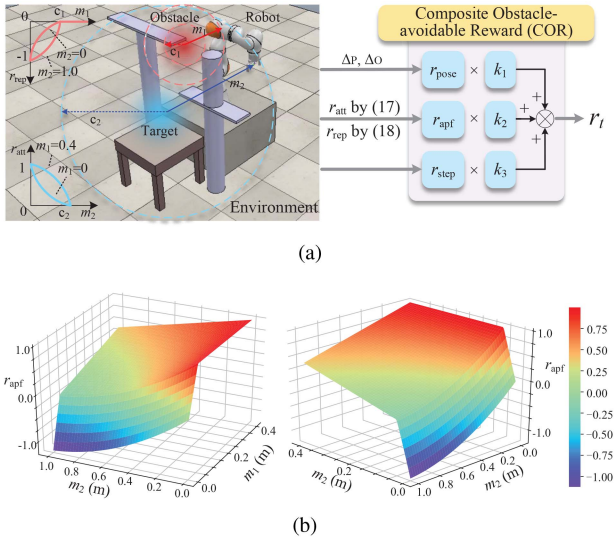


Fig. 3. Schematic diagram of the COR. (a) Structure of the COR. (b) Mapping from m_1 and m_2 to r_{apf} from two different views.

negatively affect each other. To achieve a balance between them, we propose a reward term $\boldsymbol{r}_{\text{apf}}$ motivated by the concept of the APF, which is built by quadratic functions and consists of two normalized parts constrained to [-1, 1], namely the attraction reward $\boldsymbol{r}_{\text{att}}$ and the repulsion reward r_{rep} as follows:

$$r_{\text{apf}} = r_{\text{att}} + r_{\text{rep}}. \tag{16}$$

As shown in Fig. 3, the influence radius of the obstacle is defined as c_1, and the target influence radius is defined as c_2. The shortest distance between the robot and the obstacle is defined as m_1, and the distance between the TCP and the target is defined as m_2, where $m_1 \in [0, \mathrm{c}_2/4]$, $m_2 \in [0, \mathrm{c}_2]$.

The attraction reward r_{att} is a quadratic function passing through point $(0, 1)$ and point $(\mathrm{c}_2, 0)$, which is designed as

$$r_{\text{att}} = (1 - 4\mathrm{c}_2^{-1} m_1) m_2^2 - (\mathrm{c}_2^{-1} + \mathrm{c}_2 - 4m_1) m_2 + 1 \tag{17}$$

where it can be inferred that $r_{\text{att}} \in [0, 1]$. The closer the distance between TCP and target, the larger r_{att} is.

The repulsion reward r_{rep} is a quadratic function passing through point $(0, -1)$ and point $(\mathrm{c}_1, 0)$, which is designed as

$$r_{\text{rep}} = \begin{cases} (1 - 2\mathrm{c}_1^{-1} m_2) m_1^2 + (\mathrm{c}_1^{-1} - \mathrm{c}_1 + 2m_2) m_1 - 1, m_1 \le \mathrm{c}_1 \\ 0, \text{otherwise} \end{cases} \tag{18}$$

where it can be inferred that $r_{\text{rep}} \in [-1, 0]$. The further the distance between robot and obstacle, the smaller $|r_{\text{rep}}|$ is.

According to (16)–(18), the following properties regarding r_{apf} can be extrapolated. Suppose $m_1 \to 0$ and $m_2 \to \mathrm{c}_2$, which means TCP is far away from target but the robot's body is in close proximity to obstacles, then $r_{\text{rep}} \to -1$ (its maximum penalty) and $r_{\text{att}} \to 0$ (its minimum reward), making the penalty for the collision clearly presented on r_{apf} as $r_{\text{apf}} \approx r_{\text{rep}}$. Under such circumstances, collision avoidance is prioritized. However, if $m_1 \to 0$ and $m_2 \to 0$ ($r_{\text{rep}} \to -1$ and $r_{\text{att}} \to 1$), which means TCP has almost reached the target although the robot is in high risk of collision, then $r_{\text{apf}} \approx 0$ to strike a balance between exploration and collision avoidance.

Similarly, suppose $m_1 > \mathrm{c}_1$, which means the robot is in a safe condition (far away from obstacles), then $r_{\text{apf}} \approx r_{\text{att}}$ and action exploration is prioritized since collision is almost impossible. If $m_1 \to \mathrm{c}_1$ and $m_2 \to 0$, then $r_{\text{apf}} \approx 1$ (its maximum reward value), indicating TCP has reached the target with the robot being far away from obstacles.

Remark 4: Note that in this work, the trajectory is planned in an IK-free way mentioned in Section II. In this process, while r_{att} constantly attracts the TCP to move toward the target, r_{rep} that focuses on the relative location between obstacles and the whole robot including both the end effector and the body/link, plays an essential role to achieve sufficient collision avoidance. By contrast, in other methods [29], the collision avoidance for robot's body needs to be separately considered when mapping the planned trajectories from the Cartesian space to joint space.

To make the TCP move toward the target point by constantly decreasing the position error and the orientation error, the reward term r_{step} is given as

$$r_{\text{step}} = \phi_{\text{pose}} + \phi_{\text{orient}} \tag{19}$$

where the setups of ϕ_{pose} and ϕ_{orient} refer to [8, Eqs. (12) and (13)].

Finally, the COR is defined as

$$r_t = k_1 r_{\text{pose}} + k_2 r_{\text{apf}} + k_3 r_{\text{step}} \tag{20}$$

where k_1, k_2, and k_3 are the weight coefficients of each reward term. The reward r_t is a weighted sum of several components. Such a format is commonly used to adjust the weights among the components.

V. Overall Implementation

By combining the proposed PAC with EDS and COR, the entire IK-free framework for prompt robotic motion planning with sufficient obstacle avoidance is shown by Fig. 4 and Algorithm 2.

Before starting training, a small number of expert demonstrations that can complete the original task are collected through manual teaching or traditional planning, and the expert memory $\mathrm{M_E}$ is then expanded with the proposed xGAIL, until it is filled.

Next, i prophets are selected to cosupervise the training/retraining. For each training step, the original actor network and the prophets' actor networks output action candidates $\boldsymbol{a}_t^{\text{o}}$ and $\boldsymbol{a}_t^{\text{p}(i)}$, respectively. After calculating the Q_{value} by (4), the optimal action $\hat{\boldsymbol{a}}_t$ is obtained by (5) to execute the task, and the collision-avoidable reward is fed back by the proposed COR. Then, the transition is stored into the exploration memory $\mathrm{M_L}$ by first-in-first-out (FIFO) principle, and high-quality data are winnowed via the success-pass filter into $\mathrm{M_E}$. Then, N-size batch data are sampled by (14) to update original online networks, while the prophets' networks are updated by a delay rule. The episode is terminated when the agent reaches the target area with

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Overall Implementation of the Motion Planning Framework.

Initialize the PAC with i prophets;
Initialize the $\mathrm{M_L}$ and $\mathrm{M_E}$ for EDS, and fill up $\mathrm{M_E}$ by xGAIL;
for each episode **do**
  Randomly reset the environment.
  for each time step t **do**
    Generate multiple action candidates $\boldsymbol{a}_t^{\mathrm{o}}, \boldsymbol{a}_t^{\mathrm{p}(i)}$ by (3) and comprehensively evaluate each candidate by (4);
    Select the optimal action $\hat{\boldsymbol{a}}_t$ by (5);
    Calculate the COR $\boldsymbol{r}_t$ by (20);
    Store $(\boldsymbol{s}_t, \hat{\boldsymbol{a}}_t, \boldsymbol{r}_t, \boldsymbol{s}_{t+1})$ to $\mathrm{M_L}$;
    Sample N data from EDS by (14);
    Update original network's online critic Q_{o} and actor μ_{o} by (9) and (10), respectively;
    Optimize the original network's target networks (Q'_{o} and μ'_{o}) by soft update;
    if t mod n_f **then**
      Update prophets' online critics $Q_{\mathrm{p}i}$ and actors $\mu_{\mathrm{p}i}$ by (7) and (10);
      Optimize prophets' target networks ($Q'_{\mathrm{P}i}$ and $\mu'_{\mathrm{P}i}$) by soft update;
    if the task is implemented **then**
      Break.

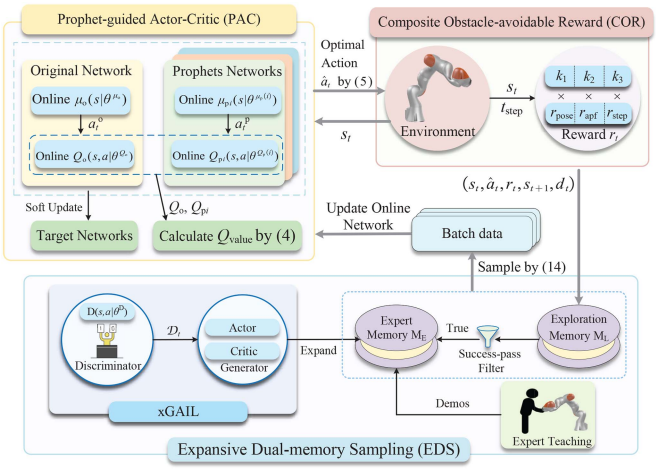


Fig. 4. Overall block diagram of the proposed framework.

a proper orientation, or when the number of episode step reaches N_{step}.

VI. Experimental Results

In this section, the proposed general framework PAC with EDS and COR (named as PEC) for collision-free robotic motion planning in different work scenes are experimentally verified with three state-of-the-art DRL algorithms, where DDPG and TD3 are based on deterministic policy, while SAC is based on stochastic policy.

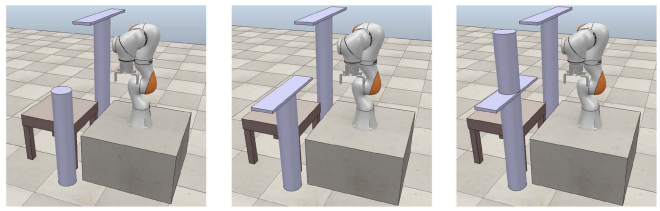


Fig. 5. Three different scenes with increasing complexity. (a) Scene 1. (b) Scene 2. (c) Scene 3.

TABLE I
Training Parameters

Parameter	Value	Parameter	Value	Parameter	Value
Actor learning rate	0.0003	N	128	$\mathrm{T_s}$	3,000
Critic learning rate	0.0003	n_f	2	$\mathrm{c_1}$	0.05
Soft update rate	0.01	λ_1	1	$\mathrm{c_2}$	1.6
Memory capacity	60,000	λ_2	0.1	k_1	1
γ	0.97	episode	1,500	k_2	0.5
φ	1/3	N_{step}	300	k_3	1

A. Experimental Setups

1) Scene Setups: As shown in Fig. 5, the aim is to train the proposed motion planning scheme to direct a 7-DoF serial robot manipulator KUKA LBR iiwa to reach a randomly given target on the table under the premise of avoiding different obstacles of concave polyhedron type. There are three scenes from low to high complexity.

2) Parameter Setups: The main parameters of the training process are shown in Table I, and all the DRL algorithms involved in the illustrative examples use the same hyperparameters unless otherwise mentioned.

3) Hardware Setups: Algorithms in this article are computed on a laptop platform equipped with Intel(R) Core(TM) i7-8750H CPU at 2.20 GHz, 16-GB RAM.

4) Software Setups: The training is performed on the CoppeliaSim software by referring to the virtual twin system in [8]. The physics engine and the time step are set as Bullet 2.83 and 50 ms, respectively.

5) Benchmark's Reward Setups: Unless otherwise illustrated, the reward function of the benchmark algorithms refers to the collision-avoidable reward (CAR) [24], which is compared with the proposed COR. For fairness, CAR comes in this way: the r_{apf} in the COR is replaced by

$$r_{\mathrm{apf}} = \begin{cases} -1 + m_1/0.05, m_1 \le 0.05 \\ 0, \text{otherwise} \end{cases} \tag{21}$$

and other reward parts in the COR remain unchanged.

The proposed COR is also compared with the motion planning reward (MPR) [4], which is defined as

$$r_{\mathrm{MPR}} = \begin{cases} \exp(-4||\boldsymbol{p}_1 - \boldsymbol{p}_0||), m_1 \le 0.05 \\ 0, \text{otherwise.} \end{cases} \tag{22}$$

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

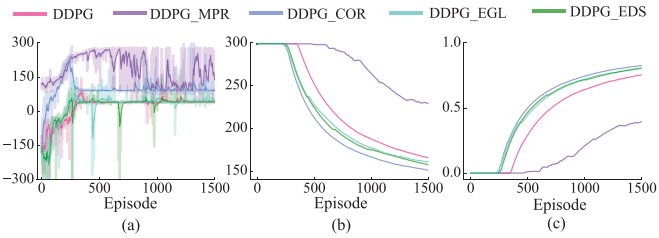


Fig. 6. Training performance comparisons in Scene 1. (a) Reward. (b) Average episode step. (c) Success rate.

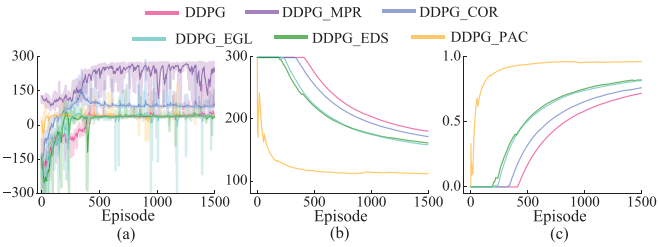


Fig. 7. Training performance comparisons in Scene 2. (a) Reward. (b) Average episode step. (c) Success rate.

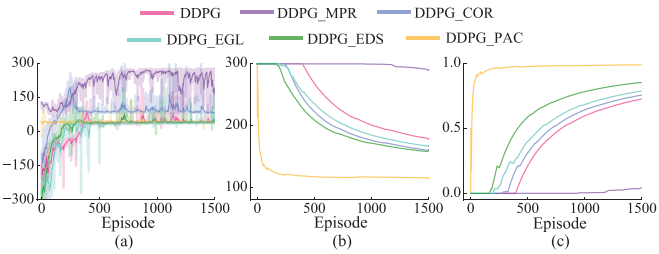


Fig. 8. Training performance comparisons in Scene 3. (a) Reward. (b) Average episode step. (c) Success rate.

B. Network Training and Contribution Verification

First, the PAC structure proposed in this article is tested by comparing the DRL algorithms (DDPG, TD3, and SAC) with their corresponding PAC-based ones (DDPG_PAC, TD3_PAC, and SAC_PAC) in different scenes. Note that Scene 1 is set as the simplest task in our experiment. Thus, PAC is only verified in Scenes 2 and 3, with the training results in Scenes 1 and 2 being the prophet in Scenes 2 and 3, respectively. Overall, the PAC plays an essential role in realizing more prompt and more stable task replanning. Take DDPG_PAC as an example, as shown in Figs. 7 and 8, the training in the two scenes can initially start with high episode rewards, and keep stable with diminutive fluctuations in the whole training process. In comparison, obvious reward fluctuations can be observed in other algorithms. Moreover, as shown in Table II, DDPG_PAC achieves much higher success rates in the training. Specifically, in the early phase (the first 500 episodes), it has reached a success rate of 94.2% and 97.6% in Scenes 2 and 3, respectively, while that of the benchmark algorithm (DDPG) only reaches 17.2% and 21.0% in the two scenes, respectively. In addition, to verify PAC's ability to be generalized to other DRL algorithms, similar

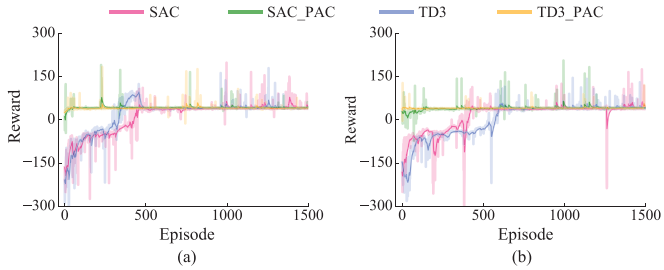


Fig. 9. Training performance of SAC and TD3 with PAC. (a) Scene 2. (b) Scene 3.

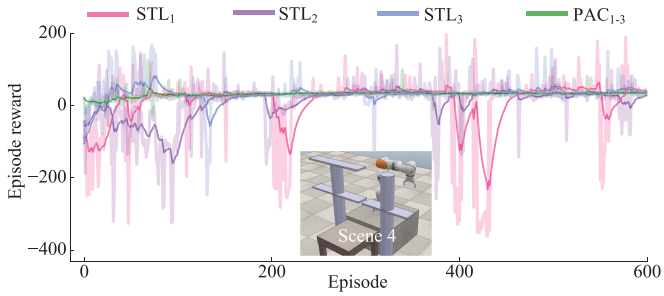


Fig. 10. Retraining performance between STL and PAC (based on DDPG).

experiments have been conducted on TD3 and SAC, the results are shown by TD3_PAC and SAC_PAC in Fig. 9 and Table II. Hence, to respond to the changes in the task scenes, the proposed PAC can significantly decrease the time costs in training DRL algorithms based on the actor–critic structure.

To further illustrate the proposed PAC's ability to integrate multiple pretrained models and its superiority in materializing more stable retraining, it is compared with the STL [28]. As shown in Fig. 10, a more complex task scene (Scene 4) is constructed, where more obstacles are added, and the target region is translated to a new place that covers more areas. The experiment is conducted based on DDPG. STL_1, STL_2, and STL_3 are transfer learning based on the training results in Scenes 1, 2, and 3, respectively, while PAC_{1-3} incorporates all the three pretrained models as prophets. It can be seen that all of STL_1, STL_2, and STL_3 show obvious fluctuations in the episode reward, even in the middle and later periods. This is caused by the sudden divergence of the trained policies. Since STL does not retain the previous policies, it usually takes a long time to make the policy converge again. According to Table III, PAC_{1-3} has the highest success rate (SR), the fewest average episode steps (n_{s}), and relatively higher average episode reward (AER) in the retraining. We use the mean square error (MSE) to quantify the fluctuations in the retraining, and PAC_{1-3} shows a significantly lower fluctuations in the episode reward ($\mathrm{MSE}_{\mathrm{R}}$) and episode step ($\mathrm{MSE}_{\mathrm{S}}$). Specifically, when comparing the differences of MSE in the whole retraining (1–600 episodes) and in the early retraining (1–200 episodes), PAC_{1-3} has much smaller differences than other counterparts, indicating the whole retraining performance is more stable.

Remark 5: In addition, as shown in Table III, we can find that the overall retraining performance is closely related to the quality

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TRAINING SUCCESS RATE COMPARISONS AMONG DIFFERENT ALGORITHMS

	Success rate (%) in Scene 1			Success rate (%) in Scene 2			Success rate (%) in Scene 3		
Training episode	1-500	501-1000	1001-1500	1-500	501-1000	1001-1500	1-500	501-1000	1001-1500
DDPG	29.2	99.0	98.4	17.2	99.6	99.2	21.0	98.0	98.4
DDPG_MPR	1.0	41.0	76.8	0.0	5.6	17.2	0.0	0.8	12.6
DDPG_COR	50.0	99.0	99.2	32.2	99.0	97.6	31.0	97.2	98.4
DDPG_EGL	46.6	97.2	98.6	50.6	97.6	97.4	39.8	96.6	99.6
DDPG_EDS	44.4	99.8	99.6	51.8	99.0	99.2	58.6	97.6	99.2
DDPG_PAC	–	–	–	94.2	97.2	98.0	97.6	99.0	99.8
TD3	24.2	99.6	99.2	8.2	100.0	99.6	0.0	78.8	99.6
TD3_PAC	–	–	–	99.4	99.2	100	99.8	100	100
SAC	35.8	99.2	99.5	9.0	96.2	99.6	15.6	99.6	99.2
SAC_PAC	–	–	–	96.0	100	100	94.6	98.8	99.8

Annotation: DDPG, TD3, and SAC are three typical DRL algorithms. MPR and COR refer to motion planning reward [4] and the proposed composite obstacleavoidable reward, respectively. EGL and EDS refer to expert-guided learning [16] and the proposed expansive dual-memory sampling, respectively. PAC refers to the proposed prophet-guided actor–critic. All the algorithms are equipped with CAR [24] except DDPG_MPR and DDPG_COR.

TABLE III
PERFORMANCE COMPARISONS BETWEEN STL AND PAC

	SR	AER	n_s	MSE_R		MSE_S	
Episode	1–600	1–600	1–600	1–600	1–200	1–600	1–200
STL_1	75.3%	10.67	156.8	88.52	69.38	66.40	74.81
STL_2	67.5%	9.06	176.6	61.56	87.34	55.97	74.73
STL_3	89.5%	33.19	156.2	33.05	48.73	39.09	56.47
PAC_{1-3}	91.0%	32.37	125.7	11.37	17.57	18.18	28.62

TABLE IV
PROPERTIES OF EGL AND EDS

	Manual participation	Data status	Efficiency
EGL	High	Static	High
EDS	Low	Dynamic	Very high

of the pretrained models. In general, Scene 3 is the most similar one to Scene 4, thus STL_3 performs better than STL_1 and STL_2. Such conclusions have been demonstrated in [28]. However, STL_2 pretrained in Scene 2, which is more similar to Scene 4 than Scene 1, actually performs worse than STL_1. In fact, it is always difficult to foresee which pretrained model matches the new task scene best, especially when the scene is complex. Regarding this, the proposed PAC based on a multiactor–multicritic structure can incorporate and refer to various pretrained models to cosupervise the retraining.

Second, the proposed EDS is evaluated in the three scenes based on DDPG. From Figs. 6–8(a), we can find EDS has obvious advantages in terms of improving convergence speed, especially in the early training period. This is because EDS can compensate for the lack of high-quality data in the memory pool by effectively generating a large number of expert data through imitating only a few human demonstrations. In addition, as shown in Figs. 6–8(c) and Table II, EDS promotes the success rate of the training significantly. Particularly, in Scenes 2 and 3, EDS increases the success rate from 17.2% and 21.0% to 51.8% and 58.6% in the first 500 episodes, respectively. The reason is that the more complex the task, the harder it is for the agent to explore sufficient high-quality data to promote the training in the early phase.

Moreover, the proposed EDS is compared with the expert-guided learning (EGL) in [16], which has also used expert data to stimulate the training of DRL. According to the DDPG_EDS and DDPG_EGL in Figs. 6–8 and Table II, the two methods have similar and close efficacy in facilitating the early training, while the former performs better than the latter in Scene 3. However, in practice, EDS has some advantages in comparison with EGL. As shown in Table IV, EDS can greatly decrease the human participation in collecting and filling up the expert memory, as it can autonomously expand the data from a few demonstrations, while EGL should be filled up purely by human demonstrations. In addition, the expert data in EDS are constantly updated as the training proceeds so that it can effectively avoid local optimal caused by limited and fixed experience data, while EGL's expert data are static. Considering the aforementioned factors, the proposed EDS is more efficient than EGL.

Third, the superiority of the proposed COR is demonstrated by comparing with the CAR in [24] and the MPR in [4]. As shown in Table II, DDPG_MPR performs much worse in the training, and is completely incompetent to the tasks in Scenes 2 and 3 with significantly low success rates. The reason is that MPR was used in [4] mainly to optimize the length of paths generated by RRT*, where the nodes are close to each other. Thus, MPR can still guide the TCP to move to the next node in the training. However, in our tasks, only the start and the target points are given, and they are quite far away from each other. Under such a circumstance, MPR cannot effectively explore the environment. In addition, the collision-avoidable term in MPR is an extremely sparse one, which is pretty hard to guide the collision avoidance in complex tasks.

In comparison, CAR is a more powerful competitor, as it is dense and we have incorporated all the terms in the proposed COR in it, except the r_{apf}. However, the proposed COR still outdoes the CAR in the early training period. As shown by Figs. 6–8(a), DDPG_COR's reward converges much faster than DDPG (which is equipped with CAR) in the first 500 training

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

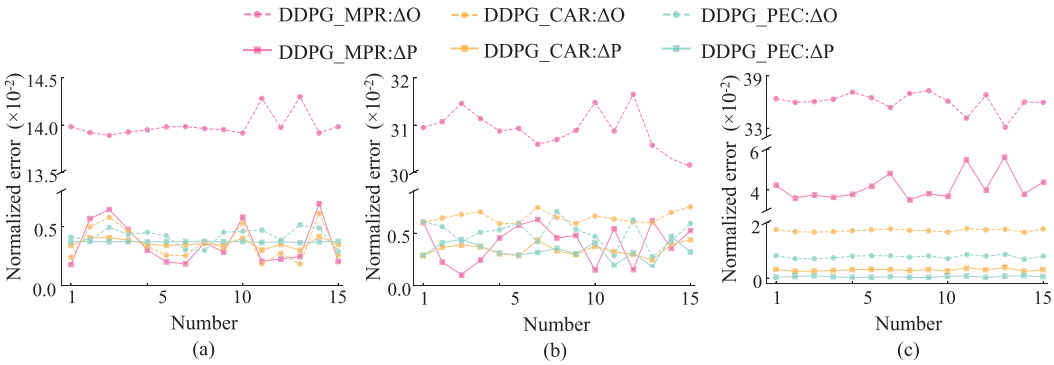


Fig. 11. Errors of different algorithms in three scenes. (a) Scene 1. (b) Scene 2. (c) Scene 3.

episodes. According to Table II, COR, benefiting from its powerful exploration ability, has promoted the success rate of DDPG in this period by 20.8%, 15.0%, and 10.0% in Scenes 1, 2, and 3, respectively. Thus, the proposed COR is a preferable reward system to realize DRL-based collision avoidance.

C. Application Instances

In this part, the feasibility of the proposed general obstacle-avoidable motion planning for robot manipulators is verified in real-world scenes. To improve the efficiency and ensure safety in training, the proposed framework PEC is trained in a virtual twin system like that in [16] and is then directly applied to real-world applications. Note that in the proposed PEC, EDS and PAC are techniques for efficient policy training, while the ultimate policy for trajectory generation is mainly determined by different reward systems. Since DDPG shows better performance than TD3 and SAC in the early training (as illustrated in Table II), it is chosen for the real-world experiments. All the algorithms are trained 1500 episodes.

First, 15 tests have been conducted in the three scenes to place blocks to random target positions on the table using DDPG_MPR, DDPG_CAR, and DDPG_PEC (with COR), respectively. The normalized position and orientation errors (Δ_{P} and Δ_{O}, defined as the scaled value of the true error) are shown in Fig. 11, and the maximum permitted errors Δ_{P} and Δ_{O} are 0.575×10^{-2} (2 cm) and 3.33×10^{-2} (6°), respectively. If both the position error and orientation error are within the maximum permitted values, the gripper will open its fingers to place the block. According to Table V, it can be seen that DDPG_MPR has large Δ_{O} in all the three scenes due to the lack of orientation control reward. Thus, it cannot satisfy our tasks and shows low success rates. DDPG_MPR is able to accomplish small Δ_{P} in Scenes 1 and 2. However, it shows large Δ_{P} in the complex Scene 3, since it is unable to balance between controlling the position and avoiding the multiple obstacles. In comparison, both the DDPG_CAR and the proposed DDPG_PEC can achieve a 100% success rate in all the scenes. The two methods have close errors in Scenes 1 and 2. However, the latter shows much smaller errors in Scene 3, indicating its superiority in handling more complex obstacle avoidance tasks. In addition, DDPG_PEC has much shorter trajectory length than DDPG_CAR in all the three scenes, which also supports its superiority.

TABLE V
PERFORMANCE COMPARISON OF TRAJECTORY GENERATION

		DDPG_CAR	DDPG_MPR	DDPG_PEC
Scene 1	**Average** $\Delta_{\mathrm{P}}(\times 10^{-2})$	0.339	0.357	0.371
	Average $\Delta_{\mathrm{O}}(\times 10^{-2})$	0.357	13.99	0.917
	Average length (m)	1.084	0.907	1.035
	Success rate (%)	100	0	100
Scene 2	**Average** $\Delta_{\mathrm{P}}(\times 10^{-2})$	0.342	0.407	0.331
	Average $\Delta_{\mathrm{O}}(\times 10^{-2})$	0.650	30.91	0.504
	Average length (m)	1.091	0.999	1.010
	Success rate (%)	100	0	100
Scene 3	**Average** $\Delta_{\mathrm{P}}(\times 10^{-2})$	0.295	4.086	0.049
	Average $\Delta_{\mathrm{O}}(\times 10^{-2})$	1.717	36.02	0.771
	Average length (m)	1.119	1.011	0.995
	Success rate (%)	100	0	100

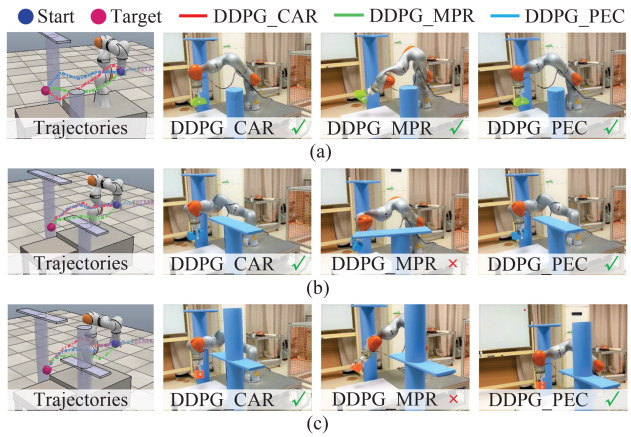


Fig. 12. Trajectory generation performance of block placing tasks. The green tick indicates success in the task. The red cross indicates failure in the task. (a) Scene 1. (b) Scene 2. (c) Scene 3.

Without loss of generality, the task of placing blocks to random target positions on the table is demonstrated in both virtual twin system and real-world applications. The planned trajectories and ultimate results of each method in each scene are presented in Fig. 12, and the variations of the normalized error during the whole task process is shown in Fig. 13. Note that since DDPG_MPR does not contain an orientation control term, the task for it is deemed successful if the TCP reaches the target point without collision (regardless of the orientation),

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

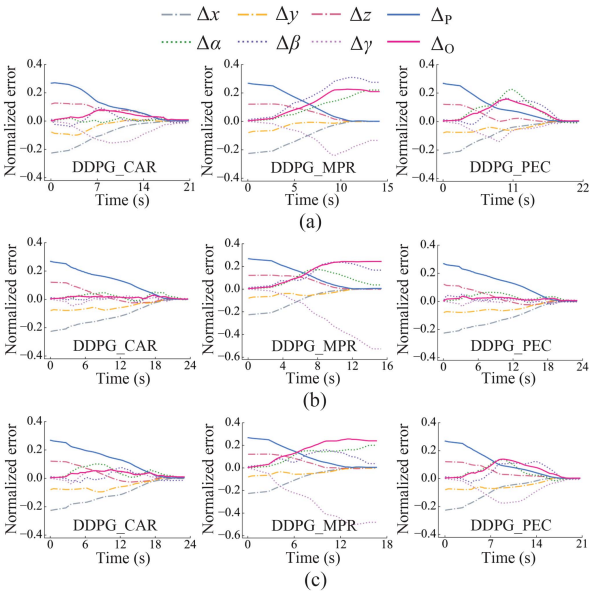


Fig. 13. Error variations of the block placing tasks. (a) Scene 1. (b) Scene 2. (c) Scene 3.

while for DDPG_CAR and DDPG_COR, a proper orientation is required. It can be seen that DDPG_MPR fails in Scene 2 due to the collision with obstacles. Specifically, the trajectory of the TCP has managed to avoid obstacles. However, the robot's body still collides while moving, which has posed serious safety problems to real-world applications. DDPG_MPR also fails in the Scene 3 as the Δ_P exceeds the maximum permitted value, although it does not collide with obstacles.

In comparison, both DDPG_CAR and the proposed DDPG_PEC can successfully accomplish all the block placing tasks. As shown in Fig. 13, both of the algorithms can approach the target by constantly decreasing the Δ_P to almost zero, but the Δ_O of DDPG_PEC is changed more frequently than that of DDPG_CAR, indicating that the former can adjust the orientation of the end effector more flexibly to avoid obstacles. Besides that, it can be seen that DDPG_PEC takes less execution time than DDPG_CAR in the task Scene 3. Thus, it can be concluded that the proposed PEC framework enables DRL to be fully competent to perform a variety of obstacle-avoidable motion planning tasks (especially complex ones) better than other counterparts.

VII. Conclusion

This article has discussed the problem of obstacle-avoidable motion planning for robots based on DRL. We have shown that the PAC structure based on the expert strategy can speed up the retraining for robotic tasks in complex task scenes and support the prompt replanning in response to changes in the scenes. In addition, to solve the inefficient learning caused by low-quality memory data in the early training period and inefficiency of obtaining expert demonstrations, an EDS mechanism has been proposed to expand the expert memory and improve the training efficiency via an increasingly unbiased dual-memory sampling rule. Besides that, to enhance the obstacle-avoidable ability of the robots, a COR system has been constructed to help to quickly generate an optimal trajectory, which enables the end effector and body/link of a robot to avoid obstacles simultaneously in a complex environment, while also strike a balance between obstacle avoidance and action exploration.

The proposed scheme is experimentally verified on a 7-DoF serial manipulator. The experimental results showed that the proposed PAC has an efficient exploration ability, and can achieve more stable and prompt retraining compared with STL. Hence, it has great adaptability to the change of scenes. Besides that, the proposed EDS was compared with EGL. The results showed that it can improve the utilization of expert demonstrations, and the success rate has been significantly promoted. Moreover, in comparison with CAR and MPR, the proposed COR can effectively ensure collision-free movement of robots in different scenes, especially in complex ones. It is worth mentioning that the proposed approach is a general solution, which can be applied to all DRL algorithms based on actor-critic, and the motion can be planned in an IK-free way without redundant work on collision avoidance for robot's body.

References

[1] J. Lv, Y. Wang, S. Wang, X. Bai, R. Wang, and M. Tan, "A collision-free planning and control framework for a biomimetic underwater vehicle in dynamic environments," *IEEE/ASME Trans. Mechatron.*, vol. 28, no. 3, pp. 1415–1424, Jun. 2023.

[2] H. Shen, W.-F. Xie, J. Tang, and T. Zhou, "Adaptive manipulability-based path planning strategy for industrial robot manipulators," *IEEE/ASME Trans. Mechatron.*, vol. 28, no. 3, pp. 1742–1753, Jun. 2023.

[3] S. Ruan, K. L. Poblete, H. Wu, Q. Ma, and G. S. Chirikjian, "Efficient path planning in narrow passages for robots with ellipsoidal components," *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 110–127, Feb. 2023.

[4] X. Li, H. Liu, and M. Dong, "A general framework of motion planning for redundant robot manipulator based on deep reinforcement learning," *IEEE Trans. Ind. Inform.*, vol. 18, no. 8, pp. 5253–5263, Aug. 2022.

[5] M. Safeea, R. Bearee, and P. Neto, "A modified DLS scheme with controlled cyclic solution for inverse kinematics in redundant robots," *IEEE Trans. Ind. Inform.*, vol. 17, no. 12, pp. 8014–8023, Dec. 2021.

[6] Z. Zhang, S. Chen, X. Zhu, and Z. Yan, "Two hybrid end-effector posture-maintaining and obstacle-limits avoidance schemes for redundant robot manipulators," *IEEE Trans. Ind. Inform.*, vol. 16, no. 2, pp. 754–763, Feb. 2020.

[7] S. D. Witte, T. V. Hauwermeiren, T. Lefebvre, and G. Crevecoeur, "Learning to cooperate: A hierarchical cooperative dual robot arm approach for underactuated pick-and-placing," *IEEE/ASME Trans. Mechatron.*, vol. 27, no. 4, pp. 1964–1972, Aug. 2022.

[8] F. Ying, H. Liu, R. Jiang, and X. Yin, "Trajectory generation for multiprocess robotic tasks based on nested dual-memory deep deterministic policy gradient," *IEEE/ASME Trans. Mechatron.*, vol. 27, no. 6, pp. 4643–4653, Dec. 2022.

[9] N. Naughton, J. Sun, A. Tekinalp, T. Parthasarathy, G. Chowdhary, and M. Gazzola, "Elastica: A compliant mechanics environment for soft robotic control," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 3389–3396, Apr. 2021.

[10] J. Xu, Z. Hou, W. Wang, B. Xu, K. Zhang, and K. Chen, "Feedback deep deterministic policy gradient with fuzzy reward for robotic multiple peg-in-hole assembly tasks," *IEEE Trans. Ind. Inform.*, vol. 15, no. 3, pp. 1658–1667, Mar. 2019.

[11] Y. Ma, D. Xu, and F. Qin, "Efficient insertion control for precision assembly based on demonstration learning and reinforcement learning," *IEEE Trans. Ind. Inform.*, vol. 17, no. 7, pp. 4492–4502, Jul. 2021.

[12] Y. Tsurumine, Y. Cui, E. Uchibe, and T. Matsubara, "Deep reinforcement learning with smooth policy update: Application to robotic cloth manipulation," *Robot. Auton. Syst.*, vol. 112, pp. 72–83, 2019.

[13] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 1–14.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.

[14] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
[15] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
[16] F. Ying, H. Liu, R. Jiang, and M. Dong, "Extensively explored and evaluated actor-critic with expert-guided policy learning and fuzzy feedback reward for robotic trajectory generation," *IEEE Trans. Ind. Inform.*, vol. 18, no. 11, pp. 7749–7760, Nov. 2022.
[17] P. Chen, J. Pei, W. Lu, and M. Li, "A deep reinforcement learning based method for real-time path planning and dynamic obstacle avoidance," *Neurocomputing*, vol. 497, pp. 64–75, 2022.
[18] Z. Bing et al., "Solving robotic manipulation with sparse reward reinforcement learning via graph-based diversity and proximity," *IEEE Trans. Ind. Electron.*, vol. 70, no. 3, pp. 2759–2769, Mar. 2023.
[19] M. Dong, F. Ying, X. Li, and H. Liu, "Efficient policy learning for general robotic tasks with adaptive dual-memory hindsight experience replay based on deep reinforcement learning," in *Proc. 7th Int. Conf. Robot., Control Automat.*, 2023, pp. 62–66.
[20] M. Hwang, W.-C. Jiang, Y.-J. Chen, K.-S. Hwang, and Y.-C. Tseng, "An efficient unified approach using demonstrations for inverse reinforcement learning," *IEEE Trans. Cogn. Develop. Syst.*, vol. 13, no. 3, pp. 444–452, Sep. 2021.
[21] J. Lim, S. Ha, and J. Choi, "Prediction of reward functions for deep reinforcement learning via Gaussian process regression," *IEEE/ASME Trans. Mechatron.*, vol. 25, no. 4, pp. 1739–1746, Aug. 2020.
[22] W. Xiao, L. Yuan, L. He, T. Ran, J. Zhang, and J. Cui, "Multigoal visual navigation with collision avoidance via deep reinforcement learning," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–9, 2022.
[23] F. Wang, X. Zhou, J. Wang, X. Zhang, Z. He, and B. Song, "Joining force of human muscular task planning with robot robust and delicate manipulation for programming by demonstration," *IEEE/ASME Trans. Mechatron.*, vol. 25, no. 5, pp. 2574–2584, Oct. 2020.
[24] Z. Sui, Z. Pu, J. Yi, and S. Wu, "Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 6, pp. 2358–2372, Jun. 2021.
[25] X. Li, H. Liu, X. Cheng, and M. Dong, "Prudent policy gradient with auxiliary actor in multi-degree-of-freedom robotic tasks," in *Proc. 40th Chin. Control Conf.*, 2021, pp. 3932–3936.
[26] C. Wang, C. G. Frazelle, J. R. Wagner, and I. D. Walker, "Dynamic control of multisection three-dimensional continuum manipulators based on virtual discrete-jointed robot models," *IEEE/ASME Trans. Mechatron.*, vol. 26, no. 2, pp. 777–788, Apr. 2021.
[27] W. Liu et al., "Ensemble bootstrapped deep deterministic policy gradient for vision-based robotic grasping," *IEEE Access*, vol. 9, pp. 19916–19925, 2021.
[28] T. Islam, D. M. H. Abid, T. Rahman, Z. Zaman, K. Mia, and R. Hossain, "Transfer learning in deep reinforcement learning," in *Proc. Seventh Int. Congr. Inf. Commun. Technol.*, 2023, pp. 145–153.
[29] T. Zhang, K. Zhang, J. Lin, W.-Y. G. Louie, and H. Huang, "Sim2Real learning of obstacle avoidance for robotic manipulators in uncertain environments," *IEEE Robot. Automat. Lett.*, vol. 7, no. 1, pp. 65–72, Jan. 2022.



Huashan Liu (Senior Member, IEEE) received the B.E. degree in mechanical engineering from Wuhan University, Wuhan, China, and the Ph.D. degree in mechatronics from Zhejiang University, Hangzhou, China, in 2005 and 2010, respectively.

During 2015–2016, he was a Visiting Professor with the Institute of Automatic Control Engineering (LSR), Technical University of Munich (TUM), Munich, Germany. Since 2010, he has been with the College of Information Science and Technology (CIST), Donghua University, Shanghai, China, where he is currently a Full Professor and the Vice Dean. His current research interests include robotics, artificial intelligence, mechatronics, and intelligent electrical systems.



Fengkang Ying received the B.E. degree in electrical engineering and automation from Huaqiao University, Xiamen, China, in 2019, and the M.E. degree in electrical engineering from Donghua University, Shanghai, China, in 2022. Since 2023, he has been working toward the Ph.D. degree with the Integrative Sciences and Engineering Programme (ISEP), NUS Graduate School, National University of Singapore (NUS), Singapore.

His research interests include robotics, task and motion planning, and reinforcement learning.



Rongxin Jiang received the B.E. degree in electrical engineering and automation and the M.E. degree in electrical engineering from Donghua University, Shanghai, China in 2020 and 2023, respectively.

His research interests include robotics, machine learning, and intelligent control.



Yinghao Shan (Member, IEEE) received the Ph.D. degree in electrical engineering from the Department of Electrical Engineering, The Hong Kong Polytechnic University, Kowloon, Hong Kong, in 2020 .

He is currently an Associate Professor with the College of Information Science and Technology (CIST), Donghua University, Shanghai, China. His research interests include microgrids, predictive control, and intelligent algorithms.



Bo Shen (Senior Member, IEEE) received the B.Sc. degree in mathematics from Northwestern Polytechnical University, Xi'an, China, in 2003, and the Ph.D. degree in control theory and control engineering from Donghua University, Shanghai, China, in 2011.

From 2009 to 2010, he was a Research Assistant with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong. From 2010 to 2011, he was a Visiting Ph.D. Student with the Department of Information Systems and Computing, Brunel University London, London, U.K. From 2011 to 2013, he was a Research Fellow (Scientific Co-Worker) with the Institute for Automatic Control and Complex Systems, University of Duisburg-Essen, Duisburg, Germany. He is currently a Professor with the School of Information Science and Technology (CIST), Donghua University. He has authored and coauthored around 80 articles in refereed international journals. His research interests include nonlinear control and filtering, stochastic control and filtering, as well as complex networks and neural networks.

Dr. Shen serves (or has served) as an Associate Editor or Editorial Board Member for eight international journals, including *Systems Science and Control Engineering*, *Journal of The Franklin Institute*, *Asian Journal of Control*, *Circuits, Systems, and Signal Processing*, *Neurocomputing*, *Neural Processing Letters*, and *Mathematical Problems in Engineering*. He is a Program Committee Member for many international conferences.

Authorized licensed use limited to: SHANGHAI UNIVERSITY. Downloaded on October 09,2026 at 01:44:31 UTC from IEEE Xplore. Restrictions apply.